

Role Challenge Submission Report

MeetingOps: AI-Automated Meeting Intelligence & Task Management

Candidate Name: Nikhil Sai Vempati
Submission Date: June 12, 2026
Primary Repository: [GitHub Repository Link](#)

1. Project Submission Hyperlinks

This role challenge is structured with all source code, workflows, configurations, and outputs pushed to a public GitHub repository. Below are active hyperlinks pointing directly to the components of the solution:

- * **GitHub Project Repository:** [nikhil1234108/MeetingOps-AI-Automated-Meeting-Intelligence](#)
- * **Meeting Transcript (WebVTT File):** [configs/meeting_transcript.vtt](#)
- * **Automation Source Directory:** [src/](#) Core Codebase
- * **Jira & Slack Pipeline Runner:** [workflows/run_pipeline.py](#)
- * **Created Jira Tickets (Jira Outputs Folder):** [jira_outputs](#) Folder
- * **Slack Summary Broadcasts (Slack Outputs Folder):** [Slack_Outpu_Screenshots](#) Folder

2. System Architecture & Approach

A. Multi-Format Ingestion & Sanitization:

The ingestion engine supports transcripts in WebVTT, SRT, PDF, DOCX, JSON, and TXT formats. Files are sanitized using HTML escaping to protect the downstream UI and LLM integrations from XSS or code injection payloads. The VTT parser handles block timings, speaker identifiers, and dialog normalization.

B. Structured AI Extraction (Google Gemini API):

Uses the Google Gemini API bound to Pydantic structured schemas (`MeetingAnalysisSchema`) to guarantee structured JSON output (title, date, summary, decisions list, action items). LLM errors are propagated as critical failures instead of silently falling back to mock data.

C. Human-in-the-Loop (HITL) Web Interface:

FastAPI serves a premium dashboard built using clean HTML5 and Vanilla CSS. The UI features four sequential approval gates: Ingestion Review, AI Extraction Verification, Jira Speaker-to-User Mapping, and Slack Block Kit Broadcast Preview. Users can edit extracted titles, summaries, decisions, assignee mappings, and ticket metadata (e.g., priorities or ticket types) prior to external synchronizations.

D. State Persistence & Safety Signatures:

Migrated the checkpoint context manager from SQLite to PostgreSQL (`AsyncPostgresSaver`) to support enterprise-grade concurrency and thread safety. Transcripts and state checkpoint tables are tracked using SHA-256 fingerprint caching. To ensure data integrity, state payloads are signed and verified using an HMAC-SHA256 signature to block client-side tampering of approved details.

E. Robust Integration Adapters:

- Jira Adapter: Leverages a `requests.Session` mounted with urllib3 retry adapters to automatically recover from rate limits (HTTP 429) and transient 5xx server issues using exponential backoff.
- Slack Adapter: Broadcasts rich Block Kit layouts containing clickable ticket links. Resolves speakers to their Slack handles using a local `slack\_mappings.json` file to safely mention assignees without requiring broad directory-read scopes.

F. Cyclic Feedback Loops & Action Item Reranking:

Implemented a fully cyclic LangGraph workflow. If the user provides corrections/feedback, the state machine loops back to the AI extraction node for regeneration (enforcing a 3-attempt safety limit). Sync failures also loop back to the mapping step. Action items are automatically and deterministically reranked by priority and confidence score descending.

3. Design Assumptions & Edge Cases Handled

- * GDPR Directory Email Hiding: Atlassian Jira Cloud V3 hides user emails by default (``no-email@atlassian.com``). To circumvent this, the engine performs case-insensitive display name matching (e.g. mapping transcript 'Priya' to `displayName 'Priya'`) to identify account IDs.
- * Slack Block Limits: Slack Block Kit text fields crash if a block exceeds 3000 characters. To prevent crashes from long error details, error descriptions are safely truncated to 100 characters.
- * Environment Fallback: If ``HMAC_SECRET_KEY`` is omitted from ``.env`` in production, the engine dynamically generates a cryptographically secure ``secrets.token_bytes(32)`` sequence for the active session, resolving security audit concerns of hardcoded default secrets.

4. Future Enhancements Given More Time

1. Dynamic Jira Directory Synchronization: Implement a background worker to periodically poll the Jira Users API and sync display names/emails into local configs.
2. OAuth 2.0 Integration: Replace basic authorization tokens with proper user-authorized OAuth flows for Slack and Atlassian, securing tenant access.
3. Native Voice Transcription: Add direct support for MP3/WAV/MP4 file uploads in the ingestion gate, using Gemini's native multimodal audio capabilities to transcribe and summarize in one step.

Submitted by Nikhil Sai Vempati - Lead Software Engineer